

بسم الله الرحمن الرحيم

تمرین دوم درس سیستم‌های توزیع شده

سید محمد جزایری – 810101399

فهرست

۱. مقدمه و تعریف RPC	2
۲. مفهوم شفافیت (Transparency) در RPC	2
۳. تفاوت‌های بنیادین فراخوانی محلی (Local) و راه دور (Remote)	3
۴. نقش و وظایف Server Stub و Client Stub	3
۵. مفهوم Serialization و Deserialization	4
۶. زبان تعریف رابط (IDL)	4
۷. مدیریت خطا در RPC	4
۸. مقایسه RPC و REST	5
۹. مقایسه فناوری‌های RPC	5
۱۰. پاسخ به سوالات تحلیلی تمرین	7
۱۱. جدول کاربردی (Summary Table)	7

۱. مقدمه و تعریف RPC

RPC (Remote Procedure Call) یک پروتکل ارتباطی است که به یک برنامه کامپیوتری اجازه می‌دهد تا یک زیرروال (Subroutine) یا تابع را در فضای آدرس دیگری (معمولاً در کامپیوتر دیگری در شبکه) اجرا کند. هدف اصلی RPC این است که برنامه‌نویس بدون نیاز به درگیر شدن با جزئیات برنامه‌نویسی شبکه، بتواند از سرویس‌های راه دور استفاده کند. در این مدل، فراخوانی یک تابع راه دور از نظر نحوه نگارش، دقیقاً مشابه فراخوانی یک تابع محلی (Local) است.

۲. مفهوم شفافیت (Transparency) در RPC

۲.۱ چرا RPC شبیه فراخوانی محلی به نظر می‌رسد؟

طراحان سیستم‌های RPC از لایه‌ای به نام **Stub** استفاده می‌کنند. کلاینت به جای تماس مستقیم با شبکه، تابعی را در کلاینت-استاب صدا می‌زند که دقیقاً همان نام و پارامترهای تابع اصلی را دارد. این لایه‌ی انتزاعی باعث می‌شود که برنامه‌نویس تصور کند در حال کار با یک کتابخانه محلی است.

۲.۲ خطرات و چالش‌های شباهت RPC به فراخوانی محلی

اگرچه این شباهت فرآیند توسعه را ساده می‌کند، اما می‌تواند گمراه‌کننده باشد زیرا:

۱. **تأخیر (Latency):** یک فراخوانی محلی در حد نانو ثانیه انجام می‌شود، اما RPC ممکن است میلی‌ثانیه‌ها طول بکشد.
۲. **عدم قطعیت:** در فراخوانی محلی، یا تابع اجرا می‌شود یا برنامه کرش می‌کند. در RPC، ممکن است شبکه قطع شود، سرور خاموش باشد یا پاسخ در راه بازگشت گم شود.
۳. **مدیریت حافظه:** در فراخوانی محلی می‌توان از اشاره‌گرها (Pointers) استفاده کرد، اما در RPC انتقال آدرس حافظه معنا ندارد و تمام داده‌ها باید کپی شوند.

۳. تفاوت‌های بنیادین فراخوانی محلی (Local) و راه دور (Remote)

تفاوت این دو نوع فراخوانی در چهار محور اصلی خلاصه می‌شود:

1. فضای آدرس (Address Space): در فراخوانی محلی، هر دو تابع در یک فضای حافظه هستند؛ اما در RPC، تابع کلاینت و سرور در دو ماشین کاملاً مجزا با حافظه‌های متفاوت قرار دارند.
2. نحوه انتقال پارامتر: در مدل محلی می‌توان از اشاره‌گر (Pointer) برای ارجاع به یک محل حافظه استفاده کرد، اما در RPC تمامی داده‌ها باید به صورت فیزیکی کپی و ارسال شوند (Pass by Value).
3. مدل شکست (Failure Model): در فراخوانی محلی، شکست فقط ناشی از باگ برنامه است. در RPC، "شکست جزئی" (Partial Failure) وجود دارد؛ یعنی ممکن است شبکه قطع شود در حالی که سرور هنوز در حال کار است.
4. تأخیر (Latency): زمان اجرای یک فراخوانی راه دور به دلیل سربار شبکه و فرآیند تبدیل داده، چندین مرتبه بزرگتر از فراخوانی محلی است.

۴. نقش و وظایف Client Stub و Server Stub

در معماری RPC، دو مولفه واسط برای پنهان‌سازی پیچیدگی شبکه وجود دارند:

- **Client Stub:**
 - در سمت کلاینت قرار دارد و مانند یک تابع محلی ظاهر می‌شود.
 - پارامترها را بسته‌بندی می‌کند. (Marshaling)
 - درخواست را به لایه انتقال (Transport) تحویل می‌دهد.
 - منتظر پاسخ می‌ماند و نتیجه را از حالت بسته‌بندی خارج کرده و به برنامه برمی‌گرداند.
- **Server Stub (Skeleton):**
 - در سمت سرور پیام را از شبکه دریافت می‌کند.
 - پارامترها را باز می‌کند. (Unmarshaling)

- تابع واقعی را در سمت سرور فراخوانی می‌کند.
- نتیجه تابع را مجدداً بسته‌بندی کرده و برای کلاینت ارسال می‌کند.

۵. مفهوم Serialization و Deserialization

برای انتقال داده در شبکه، اشیاء و ساختارهای داده‌ای که در حافظه رم (RAM) هستند باید به فرمتی قابل انتقال تبدیل شوند:

- **Serialization (Marshaling):** فرآیند تبدیل ساختار داده‌های پیچیده (مانند یک Object یا Struct) به یک رشته از بایت‌ها (Byte Stream) جهت ارسال از طریق شبکه.
- **Deserialization (Unmarshaling):** فرآیند معکوس در سمت مقصد، که بایت‌های دریافت شده را مجدداً به ساختار داده قابل فهم برای زبان برنامه‌نویسی تبدیل می‌کند.

۶. زبان تعریف رابط (IDL)

IDL (Interface Definition Language) زبانی مستقل از زبان برنامه‌نویسی است که برای توصیف رابط (Interface) یک سرویس به کار می‌رود. IDL مشخص می‌کند که چه متدهایی وجود دارند، چه پارامترهایی می‌گیرند و چه نوع داده‌ای برمی‌گردانند.

- **اهمیت IDL:** به کمک IDL می‌توان برای زبان‌های مختلف (مثل Go, Python, Java) کدهای رابط (Stub) تولید کرد تا سیستم‌های ناهمگن بتوانند با هم صحبت کنند.

۷. مدیریت خطا در RPC

در سیستم‌های توزیع‌شده، خطاهایی رخ می‌دهد که در برنامه‌های تک‌سیستمی وجود ندارند. مهم‌ترین این خطاها عبارتند از:

- **Timeout:** زمانی که کلاینت پاسخی در زمان مقرر دریافت نمی‌کند.

- **Server Crash**: سرور در میانه انجام عملیات متوقف می‌شود.
- **Network Partition**: شبکه بین کلاینت و سرور قطع می‌شود.
- **Message Duplication**: به دلیل تکرار ارسال درخواست توسط کلاینت، سرور ممکن است یک عملیات را دوبار انجام دهد.

۸. مقایسه REST و RPC

اگرچه هر دو برای ارتباطات شبکه استفاده می‌شوند، اما تفاوت‌های بنیادینی دارند:

- مدل ذهنی: REST مبتنی بر منابع (Resources) و متدهای استاندارد HTTP است، در حالی که RPC مبتنی بر عمل (Action) و فراخوانی تابع است.
- قالب داده: REST عمدتاً از JSON یا XML استفاده می‌کند، اما RPC (به‌ویژه gRPC) از فرمت‌های باینری بهینه استفاده می‌کند.
- کاربرد: REST برای API‌های عمومی وب عالی است؛ RPC برای ارتباطات سریع بین میکروسرویس‌های داخلی (Internal Service) ارجحیت دارد.

۹. مقایسه فناوری‌های RPC

در این بخش، فناوری‌های gRPC، JSON-RPC و XML-RPC را مقایسه می‌کنیم:

۹.۱. کارایی و سربار ارتباطی (Communication Overhead)

- **gRPC**: به دلیل استفاده از فرمت باینری **Protobuf**، کمترین میزان سربار را دارد. حجم پیام‌ها بسیار کوچک است.
- **JSON-RPC**: به دلیل استفاده از متن (Text)، سربار بیشتری نسبت به gRPC دارد اما از XML سبک‌تر است.
- **XML-RPC**: بیشترین سربار را دارد؛ برچسب‌های XML حجم پیام را به شدت افزایش می‌دهند.

۹.۲. سادگی پیاده‌سازی

- **gRPC**: پیاده‌سازی آن به دلیل نیاز به نصب پروتوکول‌بافر و کامپایل فایل‌های **proto** کمی پیچیده‌تر است.

- **JSON-RPC**: بسیار ساده است؛ تقریباً در هر زبانی با یک کتابخانه **JSON** ساده قابل پیاده‌سازی است.

- **XML-RPC**: پیاده‌سازی آن متوسط است اما کار با ساختارهای پیچیده **XML** در آن دشوار است.

۳.۹. مناسب بودن برای سیستم‌های توزیع‌شده (Scalability)

- **gRPC**: بهترین گزینه برای سیستم‌های توزیع‌شده بزرگ و میکروسرویس‌ها است (به دلیل کارایی بالا و قابلیت تعریف قرارداد).

- **JSON-RPC**: برای سیستم‌های توزیع‌شده کوچک و ارتباطات ساده بین کلاینت و سرور مناسب است.

- **XML-RPC**: برای سیستم‌های مدرن پیشنهاد نمی‌شود و بیشتر در سیستم‌های قدیمی کاربرد دارد.

۴.۹. پشتیبانی از Streaming

- **gRPC**: به صورت بومی از استریمینگ یک‌طرفه و دوطرفه (**Bi-directional**) پشتیبانی می‌کند.

- **JSON-RPC**: به طور استاندارد از استریمینگ پشتیبانی نمی‌کند و مبتنی بر درخواست/پاسخ ساده است.

- **XML-RPC**: فاقد قابلیت استریمینگ است.

۵.۹. موارد استفاده رایج

- **gRPC**: میکروسرویس‌ها، ارتباطات داخلی دیتاسنترها، اپلیکیشن‌های موبایل به سرور.

- **JSON-RPC**: وب‌سرویس‌های ساده، کیف‌پول‌های ارز دیجیتال (مانند **Ethereum API**)، ابزارهای مانیتورینگ.

- **XML-RPC**: سیستم‌های مدیریت محتوا قدیمی (مثل نسخه‌های قدیمی وردپرس) و سیستم‌های **Legacy**.

۱۰. پاسخ به سوالات تحلیلی تمرین

1. چرا **RPC** فراخوانی راه دور را شبیه محلی می‌کند؟ برای ایجاد انتزاع (Abstraction) و مخفی کردن پیچیدگی‌های شبکه از برنامه‌نویس تا سرعت توسعه افزایش یابد.
2. خطر این شباهت چیست؟ نادیده گرفتن هزینه‌های شبکه (تأخیر و پهنای باند) و عدم آمادگی برای خطاهای احتمالی شبکه که در مدل محلی وجود ندارند.
3. چه خطاهایی مختص **Remote Call** هستند؟ خطاهایی مثل گم شدن بسته در شبکه، تایم‌اوت، عدم تطابق نسخه کلاینت و سرور، و خطای **Deserialization**.
4. چه زمانی **gRPC** بهتر از **REST** است؟ در سیستم‌های **Microservices** که تعداد درخواست‌ها بسیار زیاد است و تأخیر (Latency) باید در کمترین حالت ممکن باشد.
5. چه زمانی **REST** مناسب‌تر است؟ برای سرویس‌هایی که باید توسط مرورگرها یا کلاینت‌های مختلف شخص ثالث (Third-party) به راحتی استفاده شوند.

۱۱. جدول کاربردی (Summary Table)

معیار مقایسه	gRPC	JSON-RPC	XML-RPC
کارایی و سربار	بسیار عالی (باینری)	متوسط (متنی)	پایین (متنی سنگین)
سادگی پیاده‌سازی	پیچیده (نیاز به IDL)	بسیار ساده	متوسط
سیستم‌های توزیع‌شده	بسیار مناسب	مناسب برای پروژه‌های کوچک	غیرمناسب برای سیستم‌های مدرن
پشتیبانی Streaming	دارد (دوطرفه)	ندارد	ندارد
مورد استفاده اصلی	Microservices	Simple APIs / Web3	Legacy Systems